Распознавание рукописей: валидация и эксперименты

2026-11-02

Статус и план

Главная цель — распознавание Макаровки. Digital Peter нужен как дополнительные данные для обучения: размеченных строк в нём примерно в 7 раз больше. Его собственные метрики важны только как проверка, что обучение работает (с ними есть опубликованные цифры для сравнения).

Что готово

что	где	проверено
загрузка данных, разбиение Макаровки по страницам	`src/diploma/data.py`, `splits/makarov.json`	тесты
метрики и оценка любой модели	`src/diploma/evaluation.py`, `diploma eval`	тесты, прогоны E001
описание валидации	validation.md	—
E001: готовая модель Kansallisarkisto без дообучения	E001	Макаровка 19.5% CER, Peter 64%
бейзлайн CRNN + CTC и его обучение	`src/diploma/models/crnn.py`, `src/diploma/training.py`	на CPU: за 400 шагов выучивает 8 строк без ошибок
дообучение Kansallisarkisto на наших строках	`src/diploma/trocr_training.py`, `diploma train-trocr`	тесты на крошечной модели; на настоящей — шаг обучения, сохранение и оценка на CPU
прогон всех экспериментов на сервере одной командой	`scripts/server_run.sh`	только синтаксис: на GPU ещё не запускался

Не сделано: обучение на GPU, декодирование с языковой моделью, нарезка неразмеченных страниц на строки (нужна для псевдоразметки; есть готовая открытая модель, см. «Какие ещё данные»), своя трансформерная архитектура, удаление дублей из разметки.

Запуск на сервере

Сервер — одна RTX 4090 (24 ГБ) или T4 (16 ГБ). Что нужно:

- Драйвер NVIDIA версии 525 или новее. На Linux `uv sync` ставит torch 2.14.1, собранный под CUDA 12.6 (так задано в `pyproject.toml`): ему хватает такого драйвера, и он работает и на 4090, и на T4. Сборка по умолчанию, под CUDA 13, требует драйвер 580+, которого во многих

облачных образах ещё нет. Первым шагом скрипт проверяет, что torch видит карту и считает на ней.

- На картах с памятью меньше 20 ГБ скрипт сам уменьшает батч дообучения Kansallisarkisto вдвое и вдвое удлиняет накопление градиентов: на шаг оптимизатора по-прежнему 16 строк.
- Интернет: при первом запуске скачиваются пакеты (около 3–4 ГБ, в основном torch с CUDA), модель Kansallisarkisto (2.4 ГБ) и веса ResNet34 (83 МБ). Если Hugging Face с сервера недоступен, модель можно привезти из кэша ноутбука: папку `~/.cache/huggingface/hub/models--Kansallisarkisto--cyrillic-large-handwritten` скопировать в то же место на сервере.
- Данные: `data/strings` (40 МБ) и `data/digital-peter` (970 МБ). Фото страниц `data/raw` (1 ГБ) для этих экспериментов не нужны. Если данные лежат не в `data/` рядом с кодом, путь задаётся переменной `DIPLOMA_DATA`.
- Около 20 ГБ на диске: окружение, данные, модель Kansallisarkisto (2.4 ГБ) и её две дообученные копии (по 2.4 ГБ), чекпойнты CRNN (по ~45 МБ).

Код едет через git, данные — отдельно: их в репозитории нет. Это 22 тысячи файлов, и по одному они копируются долго, поэтому на ноутбуке их сначала собирают в один архив (из корня репозитория; `SERVER` — адрес сервера, как для `ssh`):

```
tar -cf data/server-data.tar -C data strings digital-peter
scp data/server-data.tar SERVER:
```

На сервере:

```
git clone <адрес репозитория> diploma && cd diploma
mkdir -p data && tar -xf ~/server-data.tar -C data && rm ~/server-data.tar
nvidia-smi    # «Driver Version» — 525 или выше; карту не занимают чужие процессы
curl -LsSf https://astral.sh/uv/install.sh | sh    # если uv ещё нет
tmux new -s htr 'bash scripts/server_run.sh; bash'
```

`tmux` нужен, чтобы прогон не оборвался вместе с SSH: отключиться — `Ctrl+B`, затем `D`, вернуться — `tmux attach -t htr`. Сначала скрипт проверяет окружение (GPU, данные, тесты) и при проблеме сразу останавливается. Потом идут эксперименты; упавший шаг не останавливает остальные. После каждой эпохи обучение печатает CER на val и сколько примерно осталось.

Назад забрать `runs/`, `logs/` и историю обучения из `checkpoints/`; сами веса — по желанию. На сервере, в `diploma`:

```
tar -cf ~/results.tar runs logs checkpoints/*/history.jsonl checkpoints/*/config.json
```

На ноутбуке, из корня репозитория:

```
scp SERVER:results.tar . && tar -xf results.tar && rm results.tar
```

Сколько это займёт

Оценка по замерам на CPU. CRNN тратит около 110 млрд операций на строку Peter за проход вперёд и назад, Kansallisarkisto — около 2.4 трлн на строку Макаровки (с учётом пересчёта активаций ради экономии памяти); 4090 в bf16 на такой сети даёт, по опыту, 60–80 трлн операций в секунду. Загрузка

картинки с аугментацией — около 11 мс на одно ядро процессора; скрипт запускает столько процессов загрузки, сколько у сервера ядер (но не больше 16). На быстрой карте загрузка и вычисления сравнимы по времени, поэтому многое зависит от процессора сервера. Строки собираются в батчи по ширине, поэтому на белые поля уходит лишь 2% вычислений. Память: батч из 32 строк занимает несколько гигабайт из 24.

шаг	**эпох**	**на эпоху, 4090**	**всего, 4090**	**всего, T4**
установка, проверки, тесты	—	—	5–10 мин	5–10 мин
E004: дообучение Kansallisarkisto на Макаровке целиком	20	~40 с	~15–20 мин	~1 ч
E004: то же, первые 6 эпох учится только энкодер	20	~35 с	~15 мин	~50 мин
E002: CRNN на Peter	60	~15 с	~15 мин	~50 мин
E003: CRNN на Макаровке с нуля	100	~3 с	~5 мин	~15–20 мин
E003: дообучение модели E002 на Макаровке	60	~3 с	~3 мин	~10 мин
E003: Peter и Макаровка вместе (Макаровка ×4)	60	~20 с	~20 мин	~1–1.2 ч
оценка на тестах	—	—	~2 мин	~5–10 мин
всего			**~1.5 ч**	**~5–5.5 ч**

Это оценка, ошибка может быть до полутора раз в обе стороны. Настоящее время на эпоху будет видно в логе после первой эпохи.

T4 медленнее 4090 примерно в 3–4 раза: тензорные ядра дают 65 трлн операций в секунду в fp16 против 165 у 4090 в bf16, память — 320 ГБ/с против 1008. bf16 на T4 нет, поэтому обучение идёт в fp16 с масштабированием потерь (`mixed_precision` в `src/diploma/training.py`). Если время аренды ограничено, хватит первых двух часов: за них досчитается E004, главный эксперимент.

Для сравнения, ноутбук (Ryzen 5 4600H, замер шага обучения на строках Макаровки): CRNN — 1.4 с на строку, около 20 минут на эпоху Макаровки; Kansallisarkisto целиком — 16 с на строку, около 3.7 часа на эпоху; с замороженным энкодером — 5.2 с на строку, около 1.3 часа на эпоху. Весь план на ноутбуке занял бы недели.

E004 идёт первым: для Макаровки это самый вероятный кандидат на лучший результат, и если сессию придётся прервать, он уже будет посчитан.

Что ждём и что делаем по результатам

Опорные точки: готовая модель без дообучения даёт на тесте Макаровки 19.5% CER (15.7% без учёта орфографии, регистра и пунктуации). На Digital Peter у CRNN в статье датасета 5.7%, у лучших решений с сильными аугментациями — около 2.5%.

1. Проверяем, что обучение работает: E002 на `peter:test`.

- 3–7% CER — норма для нашего рецепта, идём дальше.
- больше 10% — что-то не так с обучением (learning rate, число эпох, аугментации). Чиним это до того, как делать выводы по Макаровке.

2. Какая модель лучше на Макаровке: E004 против E003.

Сравниваем лучший вариант дообученной Kansallisarkisto (E004) с лучшим вариантом CRNN (E003) на `makarov:test`.

- **E004 заметно лучше**, разница больше доверительного интервала. Значит, знания большой модели о почерке эпохи важнее, чем специализированная маленькая сеть. Дальше развиваем её (см. «Что можно сделать с моделью Kansallisarkisto»), а CRNN остаётся бейзлайном.
- **Примерно одинаково.** Маленькая CRNN справляется не хуже и в 50 раз меньше, а значит, быстрее и дешевле в экспериментах. Развиваем её: языковая модель, псевдоразметка, своя архитектура.

3. Сколько осталось до цели: лучший CER на `makarov:test`.

- **Меньше 8%.** Модель уже полезна. Дальше:
  - декодирование со словарём или символьной языковой моделью (на Peter такое давало −40% ошибок);
  - псевдоразметка 520 неразмеченных страниц по статье (сначала нужна нарезка страниц на строки);
  - своя архитектура против лучшей модели в тех же условиях.
- **8–15%.** Упираемся в данные: 807 строк обучения — это мало. Дальше:
  - словарь или языковая модель при декодировании;
  - разметить ещё страниц: каждая страница — около 37 строк, то есть +4.5% к обучающей выборке;
  - добавить близкие по эпохе датасеты (см. «Какие ещё данные»).
- **Больше 15%.** Не лучше готовой модели без дообучения — значит, что-то не так с обучением. Разбираемся по `history.jsonl` и логам, прежде чем идти дальше.

4. Помогает ли Peter Макаровке. Сравниваем три варианта E003:

- дообучение с Peter или совместное обучение заметно лучше обучения с нуля, то есть разница больше доверительного интервала. Значит, чужой почерк помогает — имеет смысл добавлять ещё данные: синтетику дореформенным текстом, другие кириллические датасеты;
- разницы нет. Peter слишком далёк от Макаровки (E001 на это намекает), и силы лучше тратить на данные самой Макаровки: разметку и псевдоразметку.

5. Когда браться за псевдоразметку. Когда у лучшей модели CER на 20% самых уверенных строк `makarov:val` опустится примерно до 2–3%. Тогда добавленные строки будут почти чистыми. Сейчас у готовой модели там 9% — рано.

Что можно сделать с моделью Kansallisarkisto

Устройство модели (разобрали веса):

часть	что это	параметров
энкодер	DINOv2-large, 24 слоя; строка растягивается до 182×1022 и режется на 13×73 = 949 квадратов по 14 пикселей	304 млн
декодер	ruRoBERTa-large, обрезанная до 12 слоёв из 24	306 млн
— словари токенов на входе и на выходе	по 51.5 млн, не общие	103 млн
— cross-attention (декодер смотрит на энкодер)		50 млн

Учили её в два этапа. Сначала базовая модель `cyrillic-large-stage1`: 138.6 млн синтетических строк, нарисованных 898 кириллическими шрифтами (почти все печатные, рукописных мало). Тексты — исторические, XVIII–XX веков, вперемешку в дореформенной и современной орфографии. На картинках «Ѣ» и «і» были, а в метках их заменили на «е» и «и», так что модель научилась видеть «Ѣ» и писать «е». Затем 80 эпох на 56.6 тысячи архивных рукописных строк; первые 11 эпох декодер был заморожен, чтобы энкодер привык к рукописи и не испортил языковую модель. Токенизатор — byte-level BPE на 50 тысяч токенов от ruRoBERTa. Современный текст ему удобен («дело» — один токен), дореформенный дробится («дѣло» — три).

Идеи, по убыванию ожидаемой пользы:

1. **Дообучение целиком и по рецепту авторов.** Уже в серверном прогоне (E004).
2. **Энкодер плюс CTC вместо декодера.** Энкодер уже читает почерк эпохи. Если поставить на него посимвольную CTC-голову, модель не сможет дописать текст, которого нет на картинке (как «Маннергеймъ» в E001), «Ѣ» станет обычным символом алфавита, распознавание ускорится в несколько раз (нет пошаговой генерации), а уверенность будет по каждому символу — это удобно для псевдоразметки. Подвох: 73 столбца на строку — около двух на символ, для CTC мало. DINOv2 умеет растягивать позиционные эмбеддинги, так что вход можно сделать шире, например 182×2044 (146 столбцов). Хороший кандидат на «свою архитектуру»: сравнить в одинаковых условиях CRNN (свёртки с весами ImageNet), энкодер архивной модели с CTC и полную модель энкодер–декодер.
3. **CTC и декодер вместе**, как в распознавании речи: обе головы учатся одновременно, а при генерации CTC штрафует варианты, которых не видно на картинке. Это против галлюцинаций, но сложнее в реализации — после пункта 2.
4. **Более широкий вход сам по себе.** Модель растягивает любую строку до 182×1022. Строки Макаровки — примерно 10:1, и при сжатии до 5.6:1 буквы становятся почти вдвое уже.
5. **Токенизатор.** Дореформенные слова дробятся на редкие куски, и языковые привычки декодера тянут к современному написанию. Дообучение это отчасти лечит; добавлять новые токены — в последнюю очередь.

Какие ещё данные

Открытых размеченных строк русской рукописи XIX — начала XX века в дореформенной орфографии мы не нашли. Всё близкое по эпохе существует, но не выложено:

- **Обучающие данные Kansallisarkisto.** По новостям проекта ArchXAI — больше 1700 страниц русских документов 1809–1917 годов из архивов Финляндии (метрические книги, бумаги высших органов Великого княжества Финляндского). Не опубликованы; контакты авторов есть на карточках моделей. Можно написать и попросить.

- **Модели Transkribus** «Russian Generic Handwriting 2» (148.8 тысячи строк, конец XIX — начало XX века) и архив Рычкова (14.9 тысячи строк, 1910-е, дореформенная орфография) — есть только модели внутри платной платформы, данных нет.
- **Работа «HTR for Russian Empire Period Manuscripts»** (подана на ICLR 2026). По выдаче поиска — около 38.5 тысячи строк в дореформенной орфографии; открыты ли данные, проверить не удалось. Тоже стоит написать авторам.

Что открыто и чем можно воспользоваться:

- **Наши 520 неразмеченных страниц.** Ближе данных не бывает. Нарезать их на строки можно открытой моделью Kansallisarkisto `rfdetr-textline-textregion-detection-2xl` (Apache-2.0); готовый код конвейера «страница → строки → текст» лежит в их демо `Cyrillic-HTR-Demo`. После этого — псевдоразметка по статье.
- **Современная рукопись для предобучения:**
  - Cyrillic Handwriting Dataset — 73.8 тысячи фрагментов, CC0. Самая чистая лицензия; альтернатива или добавка к Peter для CRNN;
  - HKR — около 63 тысяч предложений, но лицензия CC BY-NC-ND и доступ по заявке;
  - School Notebooks RU — около 1.9 тысячи страниц тетрадей, разметка по словам, MIT.
- **Дореформенный печатный текст** (nevmenandr/russian-old-orthography-ocr, MIT, Викитека) — источник текстов для синтетических строк и языковой модели.

Что говорят работы про дообучение TrOCR-подобных моделей на нескольких тысячах строк исторической рукописи:

- полное дообучение работает: на 3.6 тысячи строк латыни XVI века — 2.55% CER, лучше Transkribus (Ströbel et al., 2022);
- полностью замороженный энкодер сильно хуже (+8–10 п.п. CER), а LoRA не лучше полного дообучения (Hüttner et al., 2025);
- декодер важно учить, когда правила транскрипции отличаются от тех, что были при обучении (Kohút, Hradiš, 2025). Это наш случай: модель писала «е» вместо «ѣ».

Поэтому в E004 — полное дообучение и вариант авторов (сначала только энкодер), без LoRA и без заморозки энкодера.

Орфография

Исправлять дореформенную орфографию поверх ответа модели почти бесполезно. У готовой модели это не больше 2.5 п.п. (19.5% → 17.0%, см. E001). Модели, обученные на нашей разметке, учат орфографию сами. Дореформенные словари и тексты стоит использовать по-другому: как словарь или языковую модель при декодировании, которая исправляет и ошибки чтения.

Что нашлось в открытом доступе (поиск 1 ноября 2026):

- **Конвертеры из современной орфографии в дореформенную.** Почти все слабые или закрытые. Самый осмысленный — oldrus-ispell (Perl, GPL, 2001): ставит «ѣ» по словарю, а для неоднозначных слов (все / всѣ) выдаёт все варианты. В обратную сторону есть prereform2modern от ВШЭ (Python, MIT): с ним можно получить пары «современное — дореформенное» из любого дореформенного текста.

- **Словари.** В русском Викисловаре у лемм есть дореформенное написание (параметр `дореф=`, около 105 тысяч страниц, CC BY-SA), но его придётся доставать из дампа. Ещё есть словарь Даля в дореформенной орфографии в Викитеке.
- **Тексты для языковой модели.** В Викитеке в категории «Дореформенная орфография» около 46 тысяч страниц, в том числе Военная энциклопедия Сытина 1911–1915 годов — те же годы, что Макаровка. Ещё есть датасет nevmenandr/russian-old-orthography-ocr (MIT, около 3.9 ГБ): книги XIX века в дореформенной орфографии, картинки вместе с текстом.
- **Готовые модели.** Ближайшая по домену — модель Transkribus для русской рукописи 1910-х годов (архив Рычкова, CER 4.4%), но её веса не выложены. trocr-prereform-orthography обучена на печатных книгах, не на рукописи.
- Работ, где дореформенный словарь или языковая модель при декодировании дали бы измеримый выигрыш в распознавании, мы не нашли. Значит, такой эксперимент — хороший кандидат на собственный вклад в диплом.

Как это применить: символьная n-граммная модель на текстах Викитеки и обучающих строках Макаровки плюс beam search при CTC-декодировании CRNN. Ориентир — Digital Peter, где такой приём снизил CER с 5.7% до 3.5%. Делать это имеет смысл после того, как будут результаты E003.

Валидация моделей

Все модели проверяются одинаково, чтобы цифры разных экспериментов можно было сравнивать. Модель получает картинку строки и возвращает текст, а этот текст сравнивается с разметкой. Данные, разбиение на части и формулы метрик для всех моделей общие; как модель устроена внутри, для оценки неважно. Код лежит в `src/diploma`, запуск — `uv run diploma eval`.

Данные

датасет	что это	строк	страниц
`makarov`	Макаровский волостной суд, 1913–1914	1321	36
`makarov-main`	основная колонка страницы	851	36
`makarov-short`	узкая колонка с отметками об объявлении решения	470	30
`peter`	Digital Peter: письма и бумаги Петра I, начало XVIII века	9653	662

Обе коллекции — скоропись в дореформенной орфографии: ѣ, і, ъ на конце слов. Почерк и эпоха разные: у Петра — XVIII век, только строчные буквы и почти без пунктуации. В Макаровке — канцелярская рука начала XX века, с заглавными, пунктуацией и номерами дел.

В `data/raw/makarovskaya` лежат ещё около 555 фотографий страниц без разметки. В оценке они не участвуют; это материал для псевдоразметки.

Разбиение на train, val и test

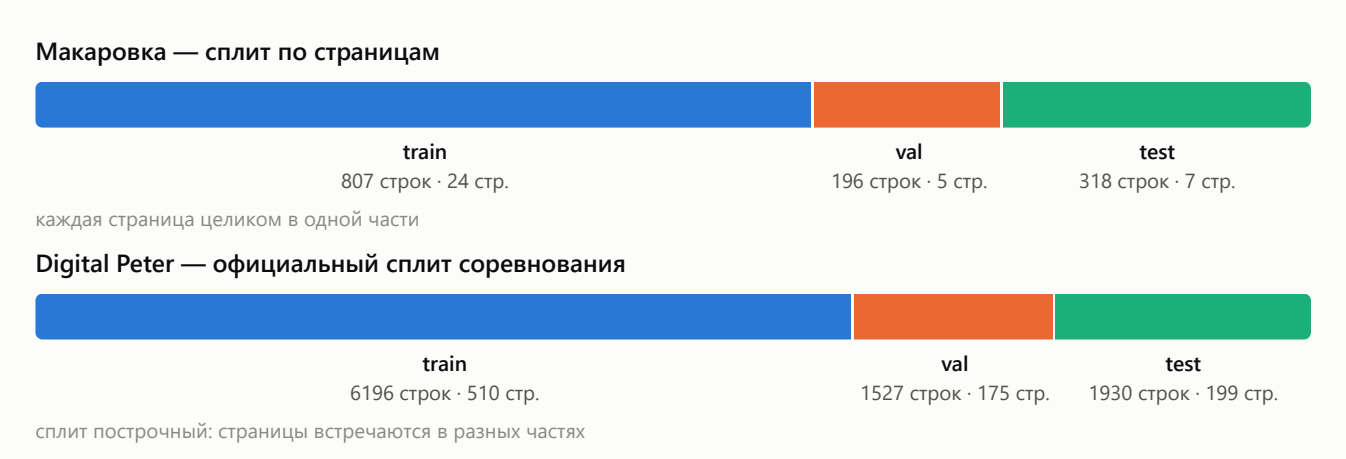


Рисунок 1. Сколько строк и страниц попало в каждую часть. Данные — загрузчики из `src/diploma/data.py`: для Макаровки разбиение читается из `splits/makarov.json`, для Peter — из `mapper.csv` соревнования. Рисунок строит `scripts/report_figures.py splits`. Он нужен, чтобы было видно, насколько маленький у Макаровки тест: 318 строк с 7 страниц.

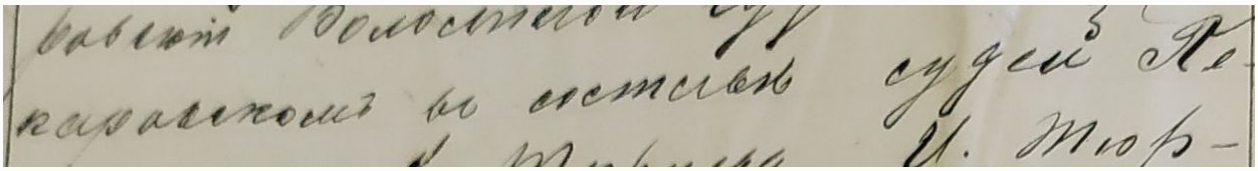
Макаровку делим **по страницам**: все строки одной страницы целиком уходят в train, val или test. Строки одной страницы написаны одной рукой, а тексты судебных записей повторяются от дела к делу («пилъ къ разбору гражданскаго», «Рѣшеніе объявлено… доволенъ»). Если делить случайно по строкам, модель видела бы в обучении соседние строки той же страницы, и CER на тесте был бы занижен.

Разбиение сделано один раз (`diploma split`, seed 0) и лежит в git. Если разметят новые страницы, загрузчик откажется работать, пока разбиение явно не пересоздадут (`diploma split --force`). После этого старые и новые цифры на тесте сравнивать нельзя, и это нужно записать в журнал экспериментов.

У Digital Peter берём **официальное разбиение соревнования**: train → train, public → val, private → test. Тогда `peter:test` — те же 1930 строк, на которых считались опубликованные результаты, и с ними можно сравниваться. Но это разбиение построчное: 242 из 662 страниц встречаются сразу в нескольких частях. Поэтому цифры на Peter получаются чуть оптимистичнее, чем были бы при разбиении по страницам.

Правило для всех датасетов: решения (гиперпараметры, эпоха, порог уверенности) принимаем по `val`, а `test` нужен только для итоговых цифр.

Как считается ошибка



эталон к а р о в с к о м ъ в ъ с о с т а в ѣ с у д е й П е

модель п а р а в с к о м ъ в ъ с о с т а в е с у д е й П е -

ошибка распознавания ошибка только в орфографии: в режиме modern не считается

CER = (3 замены + 1 вставки) / 30 символов эталона = 13.3%

в режиме modern: 2 / 28 = 7.1%

Рисунок 2. Одна настоящая строка из теста Макаровки (`main/133601-181`). Сверху — картинка строки в том виде, в каком её получает модель, вместе с краями соседних строк. Ниже — разметка и то, что прочитала модель без дообучения из эксперимента E001, выровненные по символам. Картинка взята из `data/strings/main/`, предсказание — из `runs/kansallisarkisto-zeroshot/makarov_test/predictions.jsonl`; рисунок строит `scripts/report_figures.py example`. Он показывает, что именно считает CER и чем отличается режим `modern`.

CER (character error rate) — минимальное число замен S, удалений D и вставок I символов, которое превращает ответ модели в разметку, делённое на длину разметки N:

$$\mathrm{CER} = \frac{S + D + I}{N}$$

Это та же метрика, что SER в статье Souibgui et al. и CER в соревновании Digital Peter. Пробелы тоже считаются символами. Правки и длины суммируются по всем строкам сразу, а не усредняются по строкам; иначе короткая строка весила бы столько же, сколько длинная.

WER — то же самое, но по словам. **Доля строк без ошибок** — сколько строк прочитано полностью верно.

Режимы сравнения

Одну и ту же пару «разметка — ответ модели» сравниваем с разной строгостью:

режим	что прощает	«Рѣшеніе объявлено 5/II, судъ» превращается в
`exact`	ничего	`Рѣшеніе объявлено 5/II, судъ`
`caseless`	регистр	`рѣшеніе объявлено 5/ii, судъ`
`letters`	регистр и пунктуацию	`рѣшеніе объявлено 5 ii судъ`
`modern`	ещё и старую орфографию: ѣ→е, і→и, ѳ→ф, конечный ъ	`решение объявлено 5 ии суд`

Главная цифра — `exact`. Остальные режимы показывают, откуда берутся ошибки. `modern` нужен для готовых моделей, обученных на современной орфографии: им нечестно засчитывать ошибку за «е» вместо «ѣ». Окончания вроде «-аго» → «-ого» этот режим не исправляет.

Доверительный интервал

В тесте Макаровки всего 7 страниц, и строки одной страницы зависят друг от друга. Поэтому разброс оцениваем бутстрепом по страницам: 1000 раз набираем страницы с повторениями, считаем CER и берём 2.5-й и 97.5-й перцентили. Если интервалы двух моделей сильно перекрываются, то разницу между ними мы не показали.

Уверенность модели

Если модель сообщает уверенность, сортируем строки от самых уверенных к наименее уверенным и считаем CER на верхних 10%, 20%, 50% и на всех строках. На этой проверке держится псевдоразметка из статьи: там модель размечает неразмеченные строки, и в обучение добавляются самые уверенные предсказания — 20% от размера трейна за итерацию, с порогом 0.4. Кривая показывает, насколько грязной будет такая разметка.

Частые ошибки

Отдельно сохраняется топ-30 замен, вставок и удалений символов. По нему видны систематические ошибки, например «ѣ → е».

Подготовка текста

Перед сравнением разметка и ответ модели приводятся к одному виду (`diploma.text.canonical`), чтобы различия в кодировке не считались ошибками распознавания:

- Unicode NFC: «й» из двух кодов становится одним;
- лишние пробелы схлопываются;
- кириллическая «і» заменяется на латинскую «i», потому что оба датасета размечены латинской, а готовые модели обычно пишут кириллическую;
- все виды тире заменяются на дефис;
- латинская буква внутри кириллического слова заменяется на кириллическую. Так исправляется «сторонамъ» с латинской «c»; чисто латинские слова у Петра (`piter`, `sire`) не трогаются.

Буквы ѣ, ъ, ѳ, і сохраняются: дореформенная орфография — часть задачи.

Подключение модели

Модель — любой объект с методом `predict`:

```
def predict(self, images: Sequence[PIL.Image.Image]) -> Sequence[str | Prediction]: ...
```

На вход приходят цветные картинки строк как есть, на выход — по тексту на каждую, при желании с уверенностью (`Prediction(text, confidence)`). Масштабирование и декодирование модель делает сама. Если у модели есть метод `describe()`, его ответ (чекпойнт, версия весов, параметры генерации) сохраняется вместе с метриками.

```
uv run diploma eval diploma.models.trocr:TrOCR -d makarov:test -d peter:test --name <имя
        прогона>
uv run diploma eval ... -d peter:test --sample 300 --seed 0   # случайные 300 строк
uv run diploma compare                                        # таблица по всем прогонам
```

Где лежат результаты

Каждый прогон пишет в `runs/<прогон>/<датасет>/` два файла:

- `predictions.jsonl` — для каждой строки разметка, ответ модели, уверенность и CER;
- `metrics.json` — все метрики, а также что запускали: модель и её аргументы, версия весов, выборка и seed, версии Python, torch и transformers, процессор, git-коммит, время работы.

Папка `runs/` хранится в git. Каждый эксперимент описан в `docs/experiments/`, там же журнал.

Известные проблемы разметки

- На странице `134026` (val) 11 строк размечены дважды: это одни и те же строки, вырезанные два раза (`main/134026-97…103` и `428…434`, `short/134026-108…115` и `440…447`).
- На странице `133601` (test) строка `main/133601-0` повторяет `main/133601-183`.
- В `main/133710-272` в слове «сторонамъ» стоит латинская «c»; её исправляет подготовка текста.
- 41 строка Digital Peter не входит ни в одну часть официального разбиения, мы её не загружаем.

Дубли пока не удалены: в тесте это одна лишняя строка из 318.

Журнал экспериментов

Одна строка на эксперимент. Подробности — в файле эксперимента, исходные цифры — в `runs/<прогон>/`. Как считаются метрики, описано в validation.md.

CER указан в режиме `exact`, в процентах, на тестовой части.

№	дата	что проверяли	Макаровка, test	Digital Peter, test	вывод
E001	2026-11-01	готовая модель Kansallisarkisto без дообучения	19.5	64.0 (300 строк)	как есть не годится; уверенность модели осмысленна только на Макаровке

Как добавить эксперимент

1. Прогнать модель: `uv run diploma eval ... --name <прогон>`. Результаты лягут в `runs/<прогон>/`.
2. Построить графики: `uv run python scripts/report_figures.py run <прогон> --prefix E00N`.
3. Завести `E00N-<коротко>.md`: зачем запускали, модель и версия весов, данные, точная команда, таблица метрик, графики с пояснениями, разбор ошибок, выводы.
4. Добавить строку в таблицу выше.
5. Пересобрать PDF: `uv run python scripts/build_report.py`.

E001. Готовая модель Kansallisarkisto без дообучения

Дата: 1 ноября 2026. Прогон: `runs/kansallisarkisto-zeroshot/`.

Зачем

Проверить, можно ли взять готовую открытую модель для рукописной кириллицы и сразу получить приемлемое качество на наших данных. Заодно это первая точка отсчёта: любая наша модель должна быть лучше этой.

Модель

Kansallisarkisto/cyrillic-large-handwritten — модель Национального архива Финляндии (проект ArchXAI). Среди открытых моделей, которые мы нашли, только её учили именно на исторической кириллической рукописи: 56.6 тысячи строк из архивов Эстонии, Финляндии и Латвии. Это в основном документы времён Российской империи, то есть близкий к Макаровке жанр. На своём тесте авторы получили CER 4.06% и WER 17.3%.

- Устройство: энкодер DINOv2-large для картинки и декодер ruRoBERTa-large для текста, как в TrOCR; 610.6 млн параметров. На вход — строка, растянутая до 182 × 1022 пикселей.
- В разметке, на которой её учили, «ѣ» заменили на «е», а «і» на «и». Поэтому модель пишет в современной орфографии, и честнее всего её сравнивать в режиме `modern`.
- Версия весов: `5c56397bde13f550d2ac07a7c7da61c247c17a79`.
- Декодирование — как задано в самой модели: beam search на 4 гипотезы, до 128 токенов, без повторов 3-грамм, length penalty 2. От себя мы только включили кэш декодера: на результат это не влияет, только на скорость.
- Уверенность строки — среднее геометрическое вероятностей выбранных токенов.

Код обёртки — `src/diploma/models/trocr.py`.

Данные

- **Макаровка, test**: все 318 строк с 7 страниц.
- **Digital Peter, test**: случайные 300 строк из 1930 (seed 0), со 143 страниц. Весь тест на нашем CPU занял бы около 4.5 часов, а для вывода «работает или нет» выборки хватает: доверительный интервал получился шириной около 4 п.п.

Как запускали

```
uv run diploma eval diploma.models.trocr:TrOCR -d makarov:test --batch-size 4 --name
        kansallisarkisto-zeroshot
uv run diploma eval diploma.models.trocr:TrOCR -d peter:test --sample 300 --batch-size 4 --
        name kansallisarkisto-zeroshot
```

CPU AMD (x86-64, Family 23 Model 96), Windows 11, Python 3.12.0, torch 2.14.1 (CPU), transformers 5.22.1. Скорость — около 0.12 строки в секунду, на каждый датасет ушло около 45 минут.

Что не зафиксировано:

- в репозитории ещё не было коммитов, поэтому точную версию нашего кода сохранить не удалось;
- прогон Макаровки сделан до того, как в `metrics.json` начали записывать окружение. Версии выше те же, прогоны шли на одной машине в один день.

Результаты

	строк	CER	WER	без ошибок	CER `modern`
Макаровка, test	318	**19.5** (16.8…21.8)	53.6	11.6	15.7 (12.9…18.0)
— основная колонка	183	19.2 (16.3…21.8)	50.8	7.1	16.4
— колонка отметок	135	20.6 (17.7…23.7)	62.4	17.8	12.7
Peter, test (выборка)	300	**64.0** (62.3…66.0)	100.1	3.3	61.3 (59.6…63.4)

Все числа в процентах, в скобках — 95% доверительный интервал. «Без ошибок» — доля строк, прочитанных полностью верно.

WER у Peter больше 100%, потому что вставленные моделью лишние слова тоже считаются ошибками.

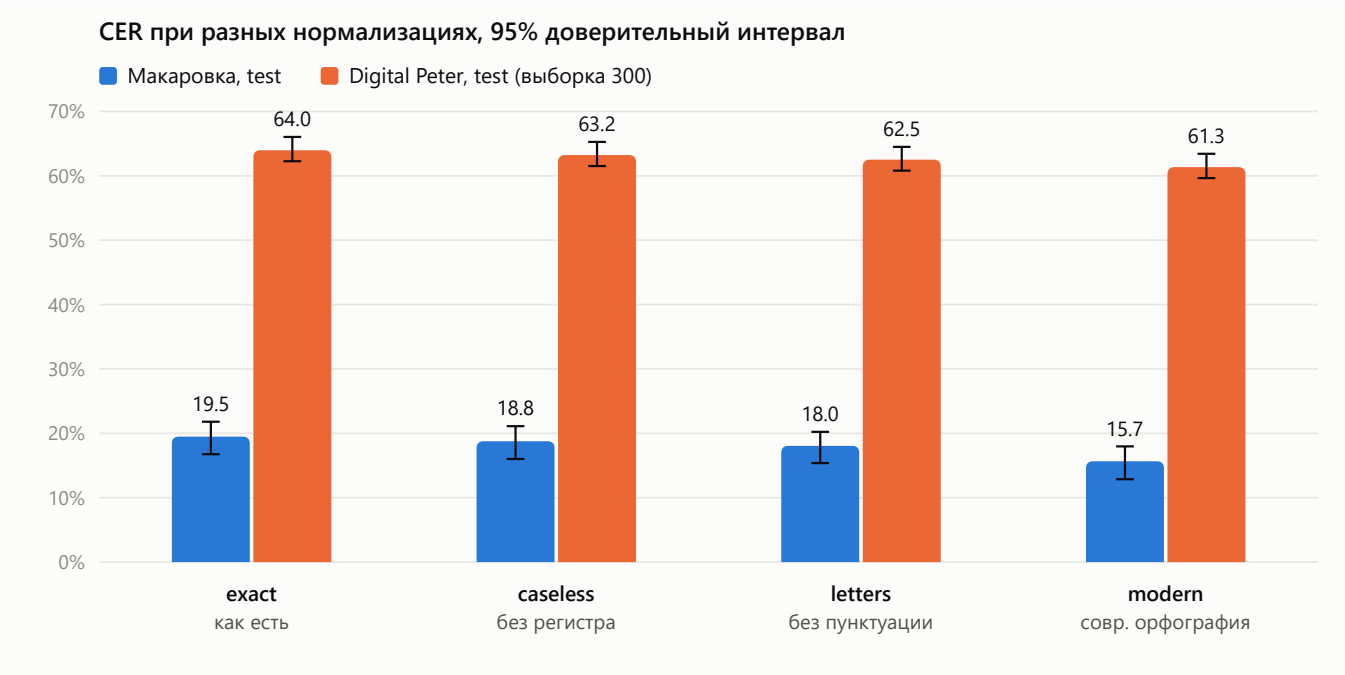


Рисунок 1. CER на обоих датасетах в четырёх режимах сравнения (что прощает каждый режим — в validation.md). Чёрные отрезки — 95% доверительный интервал бутстрепом по страницам. Данные — `metrics.json` обоих прогонов. Рисунок показывает, какая часть ошибок приходится на распознавание, а какая — на регистр, пунктуацию и орфографию.

На Макаровке переход от `exact` к `modern` снижает CER с 19.5% до 15.7%. В эти 3.8 п.п. входят регистр, пунктуация и орфография вместе; сколько приходится на каждое, показано ниже в разделе «Из чего складываются ошибки». На Peter все режимы дают примерно одно и то же: там ошибки почти целиком в чтении.

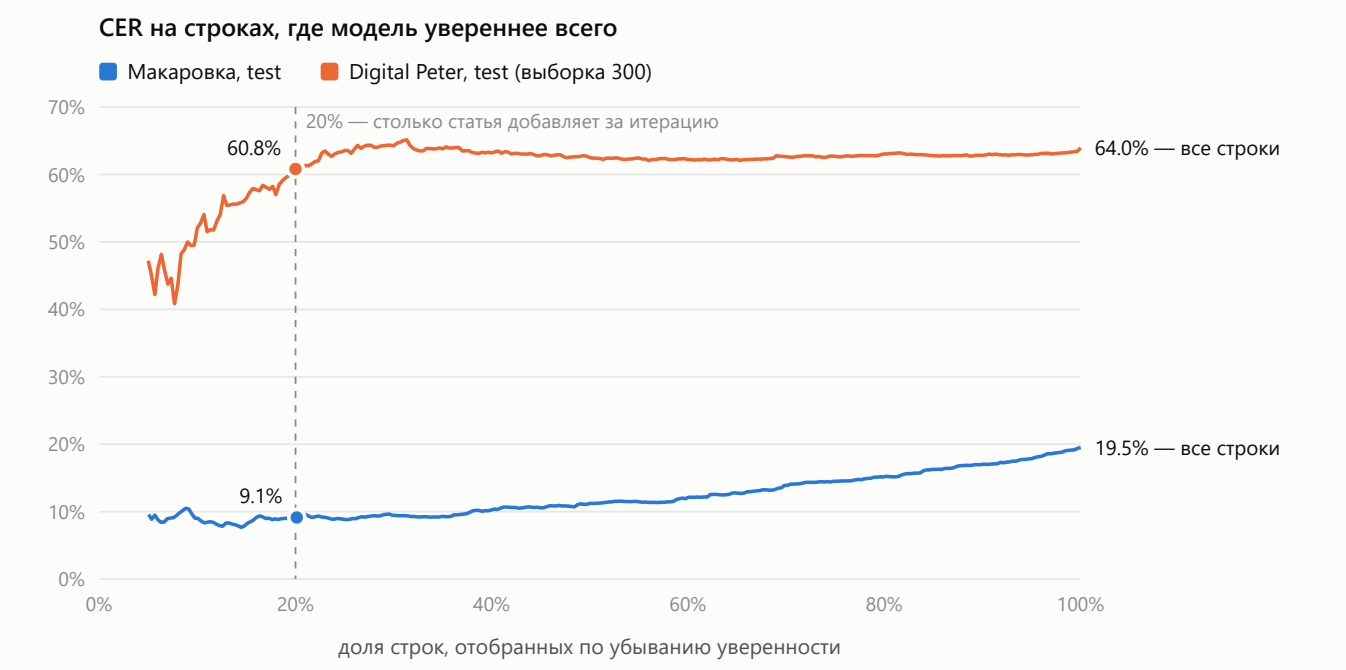


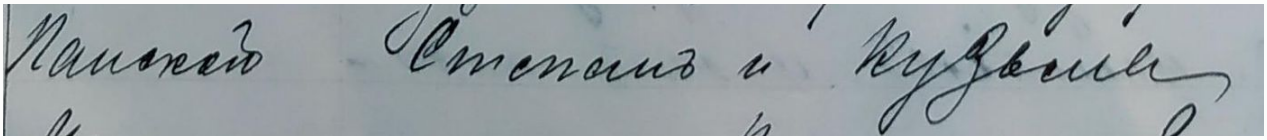
Рисунок 2. Строки отсортированы по уверенности модели, от самой уверенной к наименее уверенной. По горизонтали — какая доля строк взята, по вертикали — CER на взятых строках. Точка на 20% — столько самых уверенных предсказаний статья Souibgui et al. добавляет в обучение за одну итерацию псевдоразметки. Данные — `predictions.jsonl` обоих прогонов: уверенность и ошибки каждой строки. Рисунок проверяет, можно ли доверять уверенности модели при отборе псевдоразметки.

На Макаровке уверенность работает: на 20% самых уверенных строк CER 9.1%, на половине — 11.2%, на всех — 19.5%. Это то же, что заметили авторы статьи: когда модель уверена, она реже ошибается. Но 9% — всё ещё слишком грязная разметка, чтобы учить на ней как на правильной. На Peter уверенность почти ничего не даёт: 60.8% на верхних 20% против 64% на всех. Модель, которая плохо читает, плохо знает и то, где ошибается.

Как выглядят ошибки

Одно число CER плохо передаёт, насколько текст читаем. Поэтому ниже — настоящие строки из теста с разным CER.

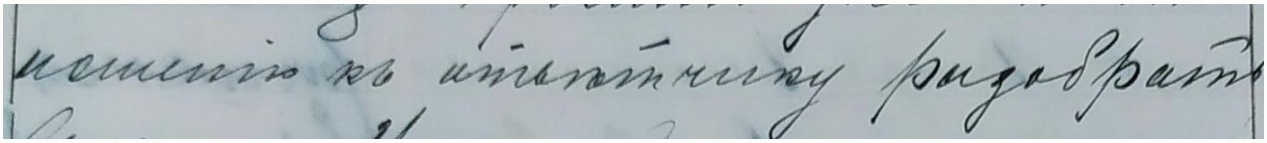
CER строки 4% 1 ошибка на 24 символа · main/134535-909 · уверенность модели 0.92



эталон П а н с к о й С т е п а н ъ и К у з ь м а

модель П а н с к о й С т е п а н ъ и к у з ь м а

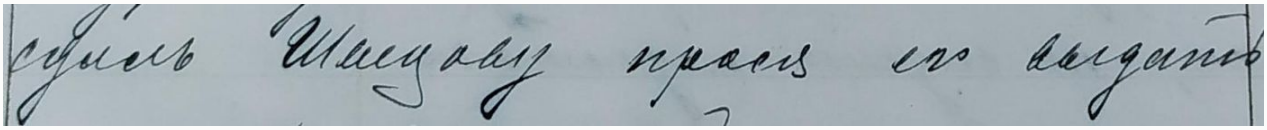
CER строки 10% 3 ошибки на 30 символов · main/134030-127 · уверенность модели 0.97



эталон н о ш е н і ю к ъ о т в ѣ т ч и к у р а з о б р а т ь

модель н о ш е н и ю к ъ о т м е т ч и к у р а з о б р а т ь

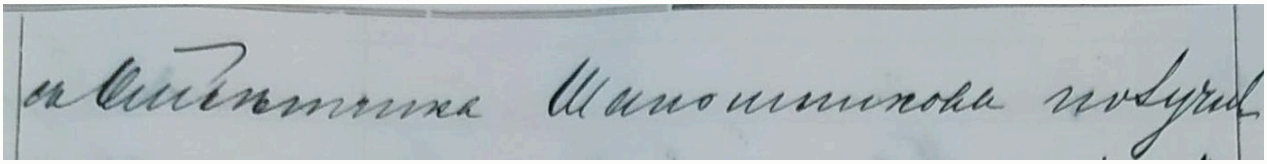
CER строки 20% 6 ошибок на 30 символов · main/134030-136 · уверенность модели 0.87



эталон с _ д а л ъ ш в е ц о в у п р о с я е г о в ы д а т ь

модель с у д и л ъ Ш е с т о в у п р о с я е г о в ы д а т ь

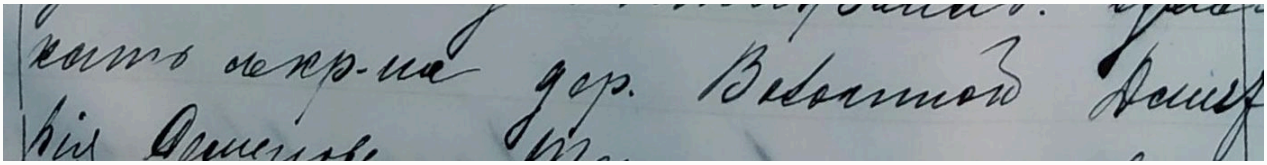
CER строки 40% 12 ошибок на 30 символов · main/134229-499 · уверенность модели 0.78



эталон с ъ о т в ѣ т ч и к а ш а п о ш н и к _ а п о л у ч и л

модель с ъ С о в е т ч и к а Ш а н о м и т к о в а п о л у ч а я

CER строки 56% 18 ошибок на 32 символа · main/134102-273 · уверенность модели 0.91



эталон к а т ь _ с ъ к р - н а д е р . В о л о г и н о _ й Д _ м и т

модель к и я _ Д е м е н о в ъ д е р . В о л о с т н о г о Д а н и я

ошибка распознавания ошибка только в орфографии: в режиме modern не считается

Рисунок 3. Строки теста Макаровки, у которых собственный CER ближе всего к 5, 10, 20, 40 и 60%. Брались только строки длиной от 15 до 38 символов: у короткого обрывка CER скачет от одной ошибки. У каждой строки — картинка, разметка и ответ модели, выровненные по символам; красным отмечены ошибки чтения, жёлтым — только орфографические. Данные — `runs/kansallisarkisto-zeroshot/makarov_test/predictions.jsonl` и картинки из `data/strings/`; рисунок строит `scripts/report_figures.py gallery`. Он показывает, что стоит за средним CER модели.

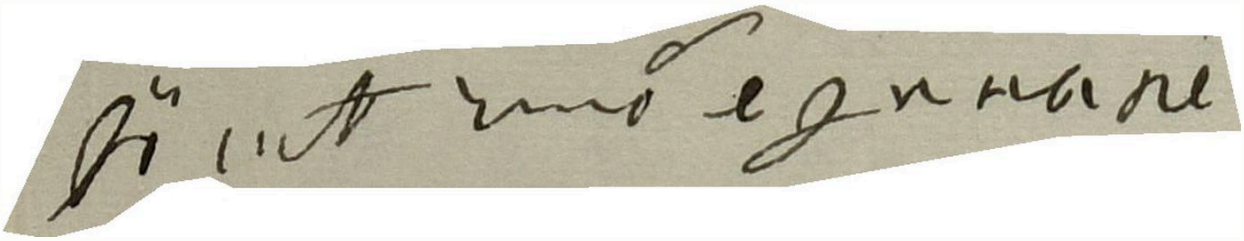
Средний CER модели на Макаровке — 19.5%, это как третья строка: около шести ошибок на строку в 30 символов. Обычные слова в таком тексте читаются, а фамилии и редкие слова нет: «сдалъ швецову» → «судилъ Шестову». На 40% искажена уже половина слов («съ отвѣтчика шапошника» → «съ Советчика Шаномиткова»), на 60% строку не понять даже с подсказкой. Видно и то, что незнакомое слово модель не читает по буквам, а подставляет похожее знакомое: «дер. Вологиной Дмит» → «дер. Волостного Дания».

CER строки 29% 5 ошибок на 17 символов · 5_21_23 · уверенность модели 0.94



эталон	н и т ␣ + к о п р е з е н т ъ
модель	н и й п о П р е з е н т ъ

CER строки 40% 8 ошибок на 20 символов · 26_4_3 · уверенность модели 0.84



эталон	в о й с к ѣ ч т о б е д и н ы ␣ п е
модель	г о ␣ и … ч т о е д и н ы е

CER строки 60% 15 ошибок на 25 символов · 26_43_10 · уверенность модели 0.83



эталон	ч и н а ч т о е ж е л и г д ѣ м ѣ ш к а т
модель	З и н а и е д е к я 1 9 0 к т о

CER строки 79% 27 ошибок на 34 символа · 25_49_4 · уверенность модели 0.66



эталон	с к о м т р а н с п о р т ѣ ␣ с к о л ␣ с и л е н ␣ + ␣ і ␣ е ж е
модель	с к о й г и л ь д и и ␣ в с п о м о г а т е л ь н ы х ъ

ошибка распознавания ошибка только в орфографии: в режиме modern не считается

Рисунок 4. То же для выборки из теста Digital Peter, уровни 20, 40, 60 и 80%. Средний CER модели здесь — 64%, то есть типичная строка — третья или четвёртая. Данные — `runs/kansallisarkisto-zeroshot/peter_test/predictions.jsonl`.

Из чего складываются ошибки

Каждую правку из выравнивания разметки и ответа отнесли к одной категории: орфография (ѣ → е, і → и, пропущенный конечный ъ), регистр (та же буква другого регистра), пунктуация и знаки, пробелы, и всё остальное — неверно прочитанные буквы.

	Макаровка	**Peter**
неверно прочитанные буквы	69.4%	79.2%
ѣ → е	7.6%	0.6%
і → и	4.5%	0.2%
пропущенный конечный ъ	0.6%	0.2%
пунктуация и знаки	7.2%	2.3%
пробелы	6.6%	16.6%
регистр	3.3%	0.8%
прочее (лишний ъ)	0.9%	0.2%
всего правок	1270	5038

Ещё две детали. 30 строк Макаровки из 318 модель заканчивает дефисом переноса, которого нет в нашей разметке: видимо, в архивах, где её учили, была другая договорённость. Канцелярские формулы она знает хорошо: «по иску кр. с. Макаровскаго» и «къ разбору гражданскаго дѣла» прочитаны без ошибок; всего таких строк в тесте 37.

Поможет ли исправление орфографии поверх модели

Модель уже пишет конечный «ъ» (250 раз при 238 в разметке) и окончания «-аго» (8 из 8). При обучении у неё заменяли только «ѣ» и «і», поэтому ровно их она и не пишет: в её ответах нет ни одной «ѣ» на 105 в разметке. Всё орфографическое вместе — 12.7% ошибок. Если бы мы исправили каждую такую ошибку идеально, CER на Макаровке снизился бы с 19.5% до 17.0%. Больше правила орфографии дать не могут: 69% ошибок — неверно прочитанные буквы, а их исправление орфографии не трогает.

Для моделей, которые мы учим сами на нашей разметке (CRNN), выигрыш будет ещё меньше: «ѣ» и «і» есть в их алфавите, и орфографии они учатся вместе с почерком. Словари и тексты в дореформенной орфографии полезнее в другом качестве — как словарь или языковая модель при декодировании. Такая модель выбирает из похожих вариантов прочтения существующее слово и этим исправляет и ошибки чтения. На Digital Peter символьная n-граммная модель снизила CER CRNN с 5.7% до 3.5% (Potanin et al., 2021).

Почему на Peter настолько хуже

1. **Другой почерк.** Модель учили на канцелярской скорописи XIX — начала XX века, той же эпохи и того же жанра, что Макаровка. Письма Петра написаны на 200 лет раньше, петровской скорописью, где многие буквы выглядят иначе (рисунок 4).
2. **Буквы над строкой.** В скорописи XVIII века часть букв выносится над строкой, а в разметке Digital Peter они записаны в строку. Модель такого письма не видела, и это одна из причин, почему на Peter пропущенные символы — 32% правок против 10% на Макаровке.
3. **Языковые привычки модели.** Когда буквы не узнаются, декодер дописывает то, что часто видел при обучении: «транспортѣ» → «гильдии вспомогательныхъ», «ісправит десантъ на острофъ» → «и законная жена его Елисавета Маннергеймъ, оба православнаго вероиспо-». Последнее — дословный кусок финских архивных документов.
4. **Пробелы.** 17% правок на Peter — пробелы: модель иначе делит слитное петровское письмо на слова.

Договорённости разметки (регистр, пунктуация, орфография) разницу не объясняют: в режиме `modern` CER на Peter всё равно 61%. Разница — в чтении: модель просто не знает этот почерк.

Выводы

1. Как есть модель не годится: 19.5% CER на Макаровке и 64% на Peter. Заявленные авторами 4% — это на их собственных архивах; другой почерк и другая нарезка строк дают разрыв в 4–5 раз даже на близком по жанру материале.
2. Как отправная точка для дообучения на Макаровке модель интересна: она уже читает почерк эпохи и знает канцелярский язык. Для Peter её знания почти не помогают.
3. Правила дореформенной орфографии поверх неё дадут не больше 2.5 п.п. (19.5% → 17.0%). Основная работа — в чтении букв, особенно фамилий.
4. Уверенность модели осмысленна на данных, похожих на её обучение, — это поддерживает идею псевдоразметки из статьи. Но с CER около 20% отбирать псевдоразметку рано: сначала нужна модель получше.

Что дальше

План экспериментов и решения по их итогам — в plan.md.